

Question de reflexion

Exercice 1:

1. Pourquoi l'algorithme $O(n^2)$ est-il acceptable pour $n \leq 1000$?

Si n est égal à 1000, il y a une somme de 1 000 000 opérations élémentaires. donc un ordinateur moderne peut exécuter facilement ce nombre d'opération pendant des millisecondes.

2. Existe-t-il un algorithme plus rapide (en $O(n \log n)$) pour la LIS ?

Oui. Au lieu de comparer chaque élément à tous les précédents, on utilise une recherche binaire pour trouver rapidement où placer chaque nouvel élément. C'est plus rapide mais plus complexe à écrire

3. Que se passe-t-il si la relation $t[j] < t[i]$ est remplacée par $t[j] \leq t[i]$?

On autoriserait les égalités : l'algorithme chercherait alors une suite "croissante ou constante" au lieu de "strictement croissante". Exemple : {3, 3, 3, 3} donnerait 4 au lieu de 1.

4. Comment adapter cet algorithme pour traiter des tableaux de long, de double, ou des objets avec un comparateur ?

- Pour long/double : juste changer le type du tableau, le reste du code ne bouge pas.
- Pour des objets : remplacer $t[j] < t[i]$ par un Comparateur (ex: `comparateur.compare(t[j], t[i]) < 0`), ce qui permet de comparer selon n'importe quel critère.

Exercice 2:

1. Comparaison des complexités :

La version naïve refait tout le tableau pour chaque élément testé $\rightarrow O(n^2)$. La version avec préfixes/suffixes ne fait que 3 passages simples dans le tableau $\rightarrow O(n)$. Elle est donc beaucoup plus rapide quand le tableau est grand.

2. Pourquoi $\text{prefixMax}[i-1] \leq t[i]$ veut dire "tout à gauche est $\leq t[i]$ " ?

Parce que $\text{prefixMax}[i-1]$ est déjà le plus grand élément à gauche. Si même le plus grand est $\leq t[i]$, alors tous les autres (plus petits) le sont aussi automatiquement.

3. Pourquoi $\text{suffixMin}[i+1] \geq t[i]$ veut dire "tout à droite est $\geq t[i]$ " ?

Même idée à l'envers : $\text{suffixMin}[i+1]$ est le plus petit élément à droite. Si même le plus petit est $\geq t[i]$, alors tous les autres (plus grands) le sont aussi.

4. Que se passe-t-il avec $< / >$ au lieu de $\leq / \geq$?

La condition devient plus stricte : les égalités ne sont plus acceptées. Par exemple, {3,

3, 3, 3} n'aurait alors plus aucun pivot, alors qu'avec $\leq / \geq$ tous les éléments internes en étaient.

5. Peut-on le faire en un seul passage, sans les deux tableaux ?

il faut connaître le minimum à droite avant de tester chaque position, donc au moins un calcul préalable reste nécessaire. On peut économiser un peu de mémoire (garder une seule variable pour le max à gauche), mais pas supprimer entièrement les deux étapes de calcul.

Exercice 5:

1. Pourquoi ne suffit-il pas de vérifier que t est une permutation de $1..n$ pour conclure qu'il s'agit d'une permutation circulaire ?

Car, une permutation veut juste dire que tous les nombres de 1 à n sont présents une fois chacun, dans n'importe quel ordre. Mais une permutation circulaire exige en plus que l'ordre $1, 2, 3, \dots, n$ soit respecté en tournant.

2. En quoi le fait de chercher la position de 1 simplifie-t-il la vérification ?

Dans $[1, 2, \dots, n]$, le 1 marque toujours le début logique. On sait direct où la séquence devrait recommencer, sans avoir à tester toutes les rotations une par une.

3. Comment modifier l'algorithme si, au lieu de $1..n$, on voulait vérifier une permutation circulaire autour d'une autre séquence (par exemple 10, 20, 30, ..., $10n$) ?

Il suffit de remplacer "valeur attendue = $k+1$ " par "valeur attendue = $(k+1) \times 10$ ", et d'ajuster la vérification de permutation en divisant chaque valeur par 10 pour retrouver son rang.

Exercice 6:

1. Pourquoi l'algorithme naïf est-il en $O(n^2)$ ou $O(n^3)$ alors que Kadane est en $O(n)$?

- **Naïf** : on teste tous les segments possibles, puis on recalcule leur somme à chaque fois $\rightarrow O(n^3)$.

- **Kadane** : on parcourt le tableau une seule fois et on réutilise `currentSum` $\rightarrow O(n)$.

2. Dans quels cas `currentSum` doit-il être réinitialisé à $t[i]$ au lieu d'ajouter $t[i]$?

On recommence à $t[i]$ quand `currentSum` < 0 .

Une somme négative ne peut que diminuer la suite : mieux vaut repartir de l'élément courant.

3. Que se passe-t-il si le tableau ne contient que des nombres négatifs ?
Pourquoi l'initialisation à $t[0]$ est-elle importante ?

On recommence à $t[i]$ quand $\text{currentSum} < 0$.

Une somme négative ne peut que diminuer la suite : mieux vaut repartir de l'élément courant.

Exercice 7:

1. Pourquoi ne peut-il pas y avoir deux éléments majoritaires différents dans le même tableau ?

Si deux valeurs distinctes a et b étaient majoritaires, chacune apparaîtrait strictement plus de $n/2$ fois. Le total des occurrences serait donc strictement supérieur à $n/2 + n/2 = n$, ce qui est impossible, puisque le tableau n'a que n cases.

4. Comment adapterais-tu la définition si l'on parlait d'élément « dominant » apparaissant au moins $n/3$ fois ? • Un seul candidat suffit-il encore ? Pourquoi ?

Un seul candidat ne suffit plus. Avec le seuil strictement supérieur à $n/3$, il peut y avoir jusqu'à 2 éléments dominants. En effet, trois éléments à plus de $n/3$ occurrences dépasseraient n au total, alors que deux ne le font pas. Il faut donc : deux candidats et deux compteurs, une annulation par triplets (trois valeurs différentes s'annulent) au lieu de paires, toujours une passe de vérification à la fin pour chaque candidat.

Exercice 8:

1. La solution avec tableau de booléens garantit de ne rater aucun élément manquant car elle marque `true` pour chaque valeur de $[1..n]$ réellement présente dans le tableau. Ensuite, elle parcourt exhaustivement toutes les valeurs k de 1 à n : si `vu[k]` est resté `false`, alors k n'a jamais été rencontré, donc il est manquant. Les doublons ne posent pas problème car ils remettent simplement `true` à une case déjà marquée.

2. Si certaines valeurs sont en dehors de $[1..n]$, plusieurs choix sont possibles :

- les ignorer, car elles ne font pas partie de l'intervalle attendu ;
- les considérer comme des données invalides et lever une erreur ou afficher un message ;
- les traiter séparément si l'énoncé le permet.

Il faut dans tous les cas éviter d'écrire `vu[x] = true` sans vérifier que $1 \leq x \leq n$, sinon on risque une erreur d'indice.

3. Une solution en place est plus économe en mémoire car elle n'utilise pas de tableau booléen supplémentaire de taille n : elle marque ou réorganise

directement le tableau d'origine, donc avec $O(1)$ mémoire auxiliaire. Ses risques sont : modification destructrice du tableau, perte des valeurs originales, code plus complexe, gestion délicate des doublons et des valeurs hors intervalle, et lisibilité/maintenabilité moins bonnes.

Exercice 9:

1. Pourquoi l'algorithme est-il en $O(n)$ pour une matrice $n \times n$ (et non pas $O(n^2)$) ?

Une seule boucle de n itérations suffit : on ne lit que $2n$ éléments (2 par ligne), pas les n^2 cases.

2. Que se passe-t-il si la matrice n'est pas carrée (par exemple 3×4) ?

Le résultat est faux ou le programme plante. Avec $n = m.length$, l'indice $n-1-i$ ne correspond plus à la vraie diagonale secondaire, et il peut sortir des limites (ArrayIndexOutOfBoundsException). Il faut donc vérifier que la matrice est carrée avant de calculer.

3. Comment adapter la fonction si l'on veut uniquement la somme de la diagonale principale ?

On supprime `sommeSecondaire` et on garde uniquement `somme += m[i][i]`.

4. Comment pourrait-on étendre cet exercice pour calculer, par exemple, la somme de toutes les diagonales parallèles à la diagonale principale ?

Pour chaque décalage k de $-(n-1)$ à $n-1$, on additionne les $m[i][i+k]$ dont les indices sont valides ($0 \leq i+k < n$). Il faut alors deux boucles imbriquées, donc $O(n^2)$.

Exercice 10:

1. Pourquoi suffit-il de choisir la somme de la première ligne comme somme de référence ?

Parce que si toutes les autres sommes sont égales à celle de la première ligne, alors elles sont toutes égales entre elles. La première ligne est juste un choix pratique.

2. Le programme vérifie-t-il uniquement la propriété "somme égale" ou aussi le fait que les nombres soient 1..9 tous distincts ?

- Si l'on souhaite imposer la condition "1 à 9 sans répétition", que faudrait-il ajouter ?

Non. Il vérifie seulement que les sommes des lignes, colonnes et diagonales sont égales.

Pour imposer **1 à 9 sans répétition**, il faut ajouter :

- vérifier que chaque valeur est entre 1 et 9 ;

- vérifier qu'aucune valeur n n'est répétée, par exemple avec un tableau booléen de taille 10.

3. Comment adapter l'algorithme pour vérifier un carré magique de taille $n \times n$ ($n > 3$) ?

- Prendre `ref = somme de la ligne 0`.
- Vérifier avec des boucles toutes les lignes, toutes les colonnes, la diagonale principale et la diagonale secondaire.
- Si toutes les sommes valent `ref`, le carré est magique.
- Pour un vrai carré magique classique, ajouter aussi la vérification des nombres $1 \dots n^2$ distincts.

4. Quelle est la complexité de l'algorithme pour une matrice $n \times n$ (en fonction de n) ?

- Temps : **$O(n^2)$** , car on parcourt les n^2 cases.
- Espace : **$O(1)$** sans vérification d'unicité.
- Avec vérification des valeurs distinctes : **$O(n^2)$** en temps et **$O(n^2)$** en espace.